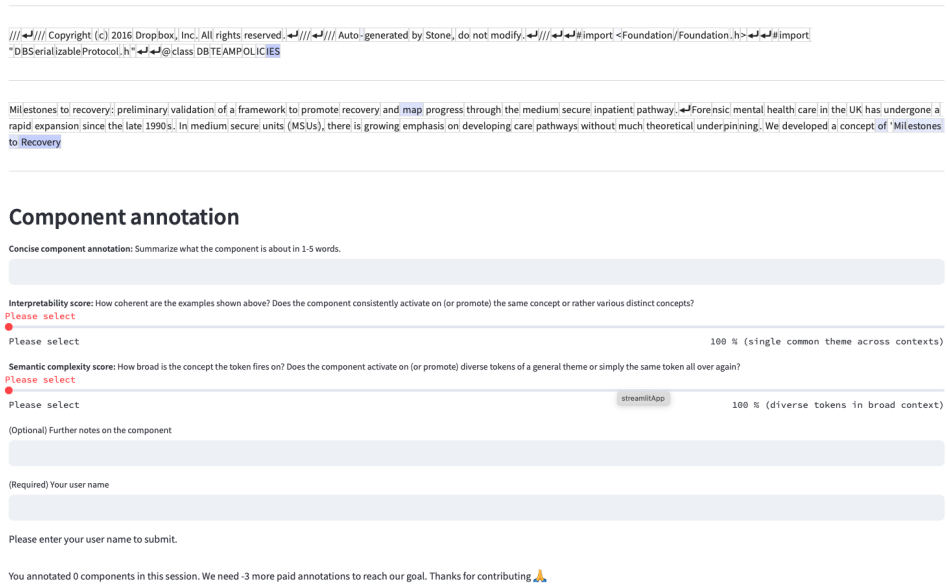

Figure 24: The human annotation interface used to obtain the interpretability ratings in Table 7. Here, we show the rating interface on the same page as the content in Fig. 23, below the example contexts. Humans were asked to write a textual description of each feature, assign a 0–100 interpretability rating, and assign a 0–100 semantic complexity rating to each feature.

See Figures 23 and 24 for examples of the human annotator interface. Humans were presented with the tokens on which the feature activated most strongly, followed by the tokens whose probabilities were most affected in Pythia-70M when the feature was ablated. This is followed by a series of example contexts in which the feature activated on some subset of tokens, where feature activations are shown in varying shades of blue (darker shades indicate higher activations). On the same page below the contexts, we ask annotators to write a textual description of the feature, and rate both its interpretability and its semantic complexity on 0–100 scales.

Crowdworkers were recruited from the ARENA Slack channel, whose members are machine learning researchers interested in AI alignment and safety. The selection of annotators certainly influenced our results; a truly random sample of human annotators would likely display higher variance when annotating features.

One common error pattern we notice is that annotators often label features according to semantic groupings (e.g., "text about politics," and do not pay attention to syntactic context (e.g., "plural nouns"). Future work could address this design bias by testing variants of the instructions.

Results of human evaluations for the Gemma Scope SAEs are described in Lieberum et al. (2024).

G Discovering LM Behaviors with Clustering

In this section, we describe our unsupervised method for discovering language model behaviors. More specifically, following Michaud et al. (2023), we cluster contexts from The Pile according to the Pythia-70M's internal state during inference. In this section, we describe our clustering pipeline and methods.

G.1 Filtering Tokens

We must first locate (context, answer) pairs for which an LM correctly predicts the answer token from the context. We select The Pile (Gao et al. (2020)) as a general text corpus and filter to pairs on which Pythia-70M confidently and correctly predicts the answer token, with cross-entropy lower than 0.1 or 0.3 nats, depending on the experiment. The model consistently achieves low loss on tokens which involve "induction" (Olsson et al., 2022)—i.e., tokens which are part of a subsequence

which occurred earlier in the context. We exclude induction samples by filtering out samples in which the bigram (final context token, answer token) occured earlier in the context.

G.2 Caching Model-internal Information

We find behaviors by clustering samples according to information about the LM's internals when run on that sample. We find clusters of samples where the model employs similar mechanisms for next-token prediction. We experiment with various inputs to the clustering algorithm:

- **Dense Activations**: We take activations (residual stream vectors, attention block outputs, or MLP post-activations) from a given context and concatenate them. To obtain a vector whose length is independent of the context length, we can either use the activations at the last N context positions before the answer token, or aggregate (sum) across the sequence dimension. We experiment with both variants.
- **Sparse Activations**: Rather than dense model activations, we can use the activations of SAE features. We concatenate and aggregate these in the same manner as for dense activations.
- **Dense Component Indirect Effects**: We approximate the indirect effect of all features on the correct prediction using 2 without a contrastive pair—namely, by setting $\mathbf{a}_{\text{patch}} = 0$. The negative log-probability of the answer token $m = -\log p(answer)$ serves as our metric for the correct prediction of the next token. The computatiom of linear effects requires saving both (1) activations and (2) gradients w.r.t m at the final N positions for each context in the dataset. We optionally aggregate by summing over all positions.
- **Sparse Indirect Effects**: Similarly, we can compute the linear effects of *sparse* activations on the correct prediction.
- **Gradient w.r.t. model parameters**: As in Michaud et al. (2023), we also experiment with using gradients of the loss w.r.t. model parameters, but with some modifications. We describe this method in more detail in §G.3 below.

G.3 Hyperparameters and Implementation Details

We apply either spectral clustering or k-means clustering. For spectral clustering, given either activations or effects x_i for sample i, we compute a matrix of pairwise cosine similarities $C_{ij} = x_i \cdot x_j/(||x_i||||x_j||)$ between all pairs of samples. Before performing spectral clustering, we normalize all elements of C to be in $[0, 1]$ by converting the cosine similarities to angular similarities: $\hat{C}_{ij} = 1 - \arccos(C_{ij})/\pi$.

We use the `scikit-learn` (Pedregosa et al., 2011) spectral clustering implementation with k-means. For all inputs except gradients w.r.t. model parameters, we used spectral clustering across 8192 samples. We chose k (the number of total clusters) to maximize the number of clusters implicated in more than one input context.

We also experimented with using gradients w.r.t. model parameters as inputs, as in Michaud et al. (2023). Here, we scale up our approach to 100,000 samples. It is intractible to perform spectral clustering given 100,000 samples, so we instead use k-means clustering. Rather than clustering the gradients themselves (which are high-dimensional), we cluster sparse random projections of the gradients down to 30,000 dimensions. When projecting, we use a matrix with entries $\{-1, 0, 1\}$. When sampling the entries of this matrix, sample a nonzero value with probability $32/30000$, and if nonzero, sample -1 or 1 with equal probability. For a sparse projection matrix with dimensions $\mathbb{R}^{n \times 30000}$, there will on average be $32 \cdot n$ nonzero entries, where n is the number of parameters in the model.[4]

H Quality of Linear Approximations of Indirect Effects

Figure 25 shows the quality of our linear approximations for indirect effects. Prior work (Nanda, 2022; Kramár et al., 2024) investigated attribution patching accuracy for IEs of coarse-grained model

[4]We only consider gradients w.r.t. non-embedding and non-layernorm parameters.

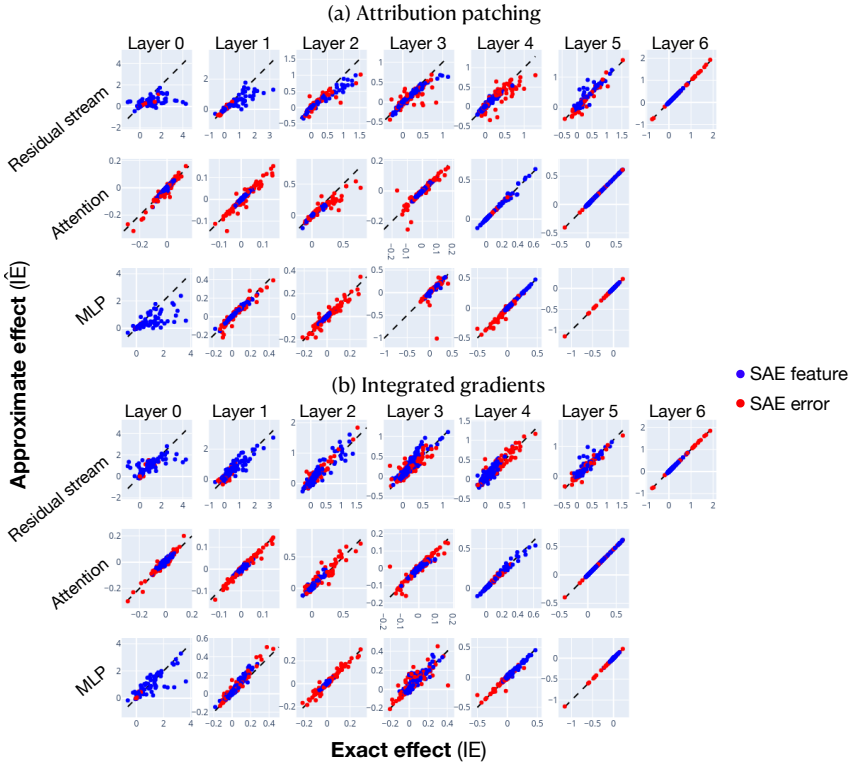

Figure 25: Approximate IEs (y-axis) and exact IEs (x-axis) using attribution patching (a; top) or integrated gradients (b; bottom). Each point corresponds to an SAE feature or SAE error at one token position of one input. Data were collected from 30 inputs from our across RC dataset.

components (queries, keys, and values for attention heads, residual stream vectors, and MLP outputs) and MLP neurons. Working with SAE features and errors, our results echo previous findings: attribution patching is generally quite good, but sometimes underestimates the true IEs. Notable exceptions are the layer 0 MLP and the residual stream in early layers. We also find that our integrated gradients-based approximation significantly improves approximation quality.